



COMP 4300

Intro to Game Programming

Lecture 22

Intro to Scripting in Game Engines
Lua Programming Language

What is Scripting?

- We have a **complex** game engine written in a performant language like C++
- We want to create / change **dynamic features** in our game without having to **recompile**
- How can we write basic code **outside** our game engine that gets executed in the game?
- **Scripting** to the rescue!

Intro to Scripting

- Scripting can be seen as a **high-level** tool that gets integrated into C++ games
- Typically handle **low-compute dynamic** and flexible aspects of gameplay
- Can also be integrated into tools such as **level editors** and game **engines**
- Scripting language is typically higher level

Scripting Benefits

- Allows **rapid** iteration and testing, making it ideal for implementing gameplay
- **Separates** logic from game engine code
- Helps **organize** code / aids scalability
 - Facilitates parallel development with engine
- Allows **designers** to **modify** behavior in an easier language outside the engine

Scripting Use Cases

- Game AI behaviors
- Dialogue systems
- Event triggers
- In-game macros
- Level editor templates
- Animation systems
- Many more

Create Macros

General Macros

Ophelliana Specific Macros

Asp...

Char...

Macro

Maly

Oh n...

She...

She...

spam

SwIn...

Trade

Trad...

UC E...

Macro

Change Name/Icon

Enter Macro Commands:

#showtooltip Steady Shot
/cast Kill Command
/cast Steady Shot
/script UIErrorsFrame:Clear()

92/255 Characters Used

Delete

New

Exit

Enter Macro Name (Max 16 Characters):

Macro

Choose an Icon:

Okay

Cancel

```
-- script a circular path
function update(entity, ellapsed_time)
    local radius = 170
    local distance = 500

    local angle = ellapsed_time * 0.09

    local x = cos(angle) * radius + distance
    local y = sin(angle) * radius + distance

    set_position(entity, x, y)
    set_rotation(entity, angle)
end
```



► Entities

▼ Level Editor

► ImGui Demo

Level path: res/levels/undead_cit

Camera: (-249.28, 528)

Mouse position: (-188, 612)

Tile edit

☐ Show grid

☐ Snap to grid



Tile position: (-12, 38)

Tile id: 19

Tileset id: 2

☐ Selection mode



Animation editor

Show grid

Snap to grid

▶ Rendering settings

res/scripts/archer.lua

Script

archer

Entity

die

Animation

▶ Sprite sheet



1

Frame

Add animation

Remove animation

Save

▼ Animation properties

name

die

looped

frameCount

13

-

+

▶ frameRects

{ unordered_map, size = 1 }

frameTime

100

-

+

(ms)

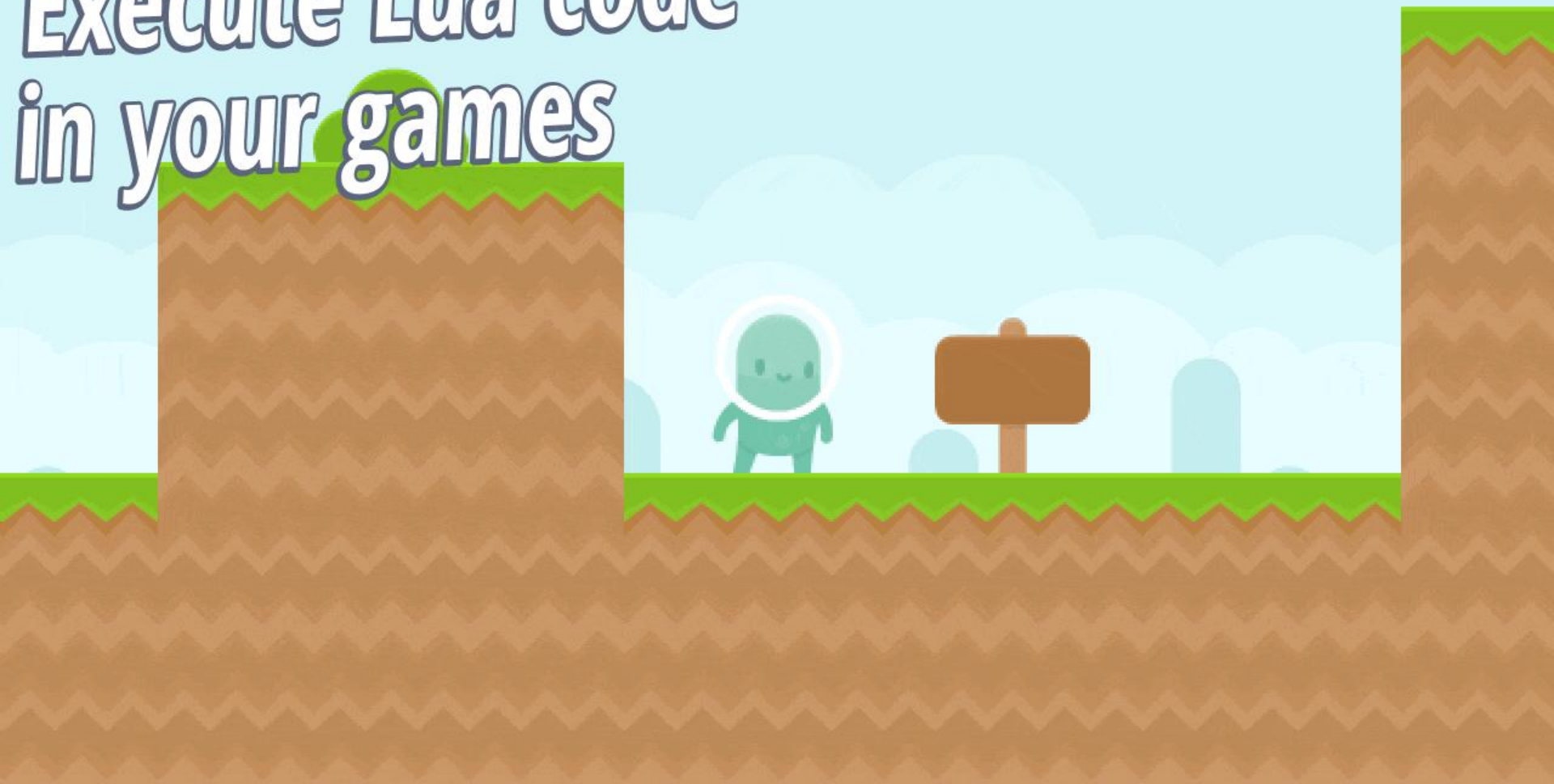
offsets

{ unordered_map, size = 0 }

sounds

{ vector, size = 0 }

Execute Lua code
in your games

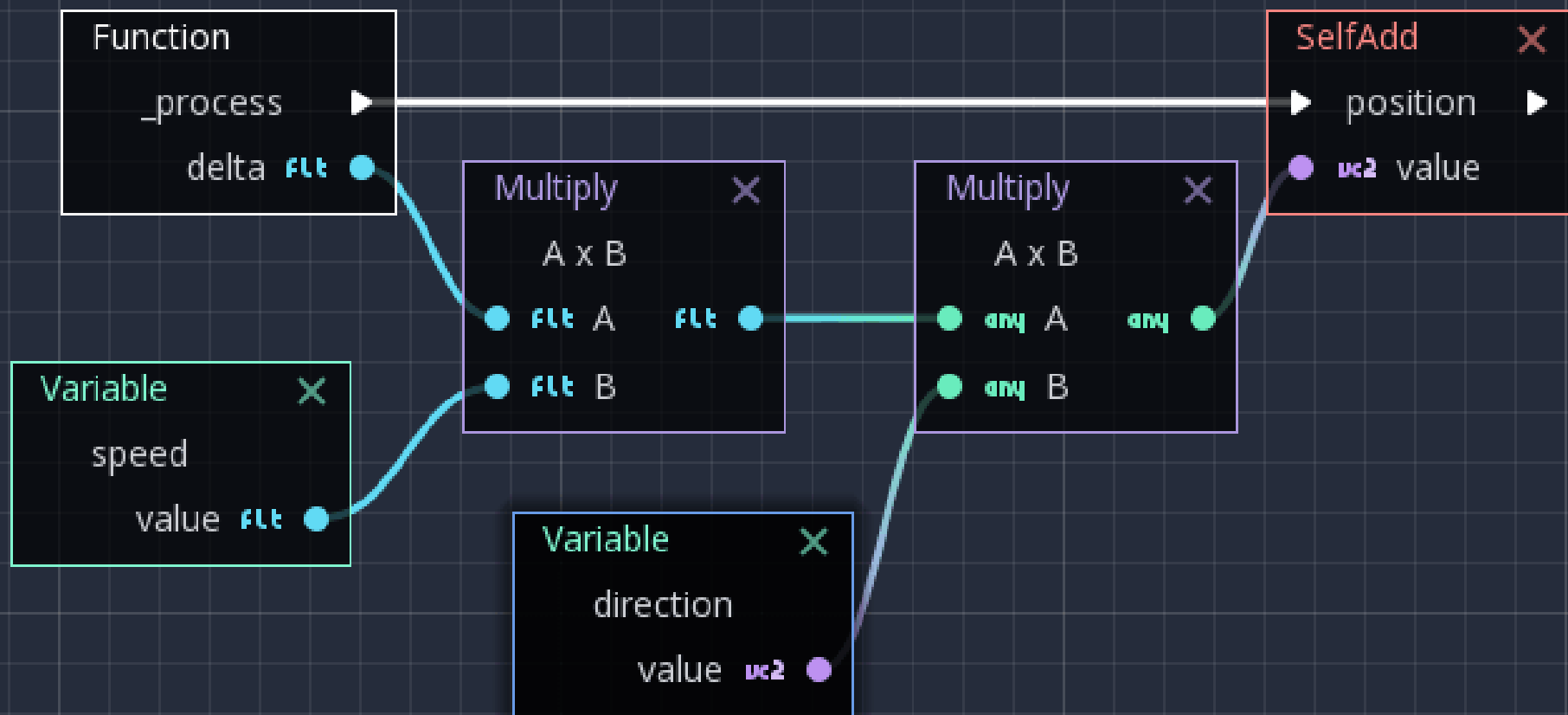


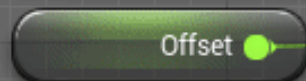
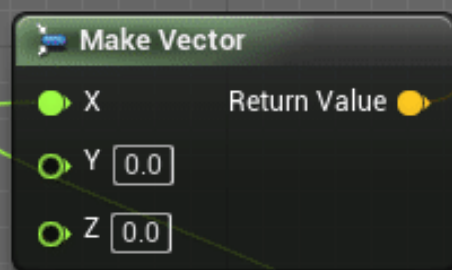
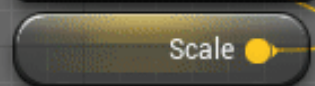
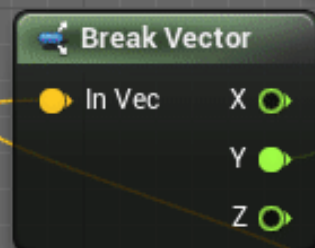
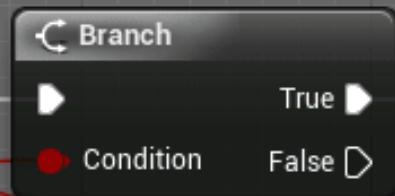
Scripting Languages

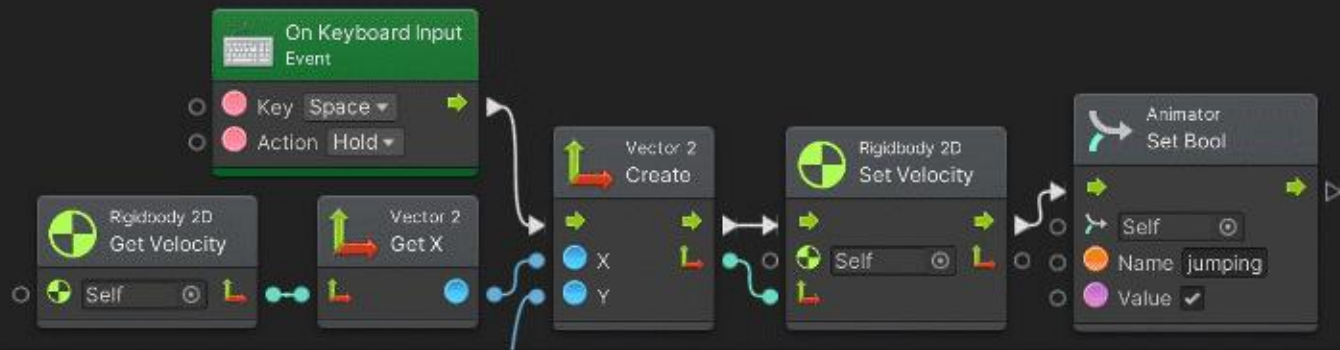
- Most scripting languages are high-level, interpreted, dynamic typing
 - Lua
 - Python
 - JavaScript
- Lua is especially popular to pair with C++ due to its efficiency, small footprint, and is highly embeddable

Visual Scripting

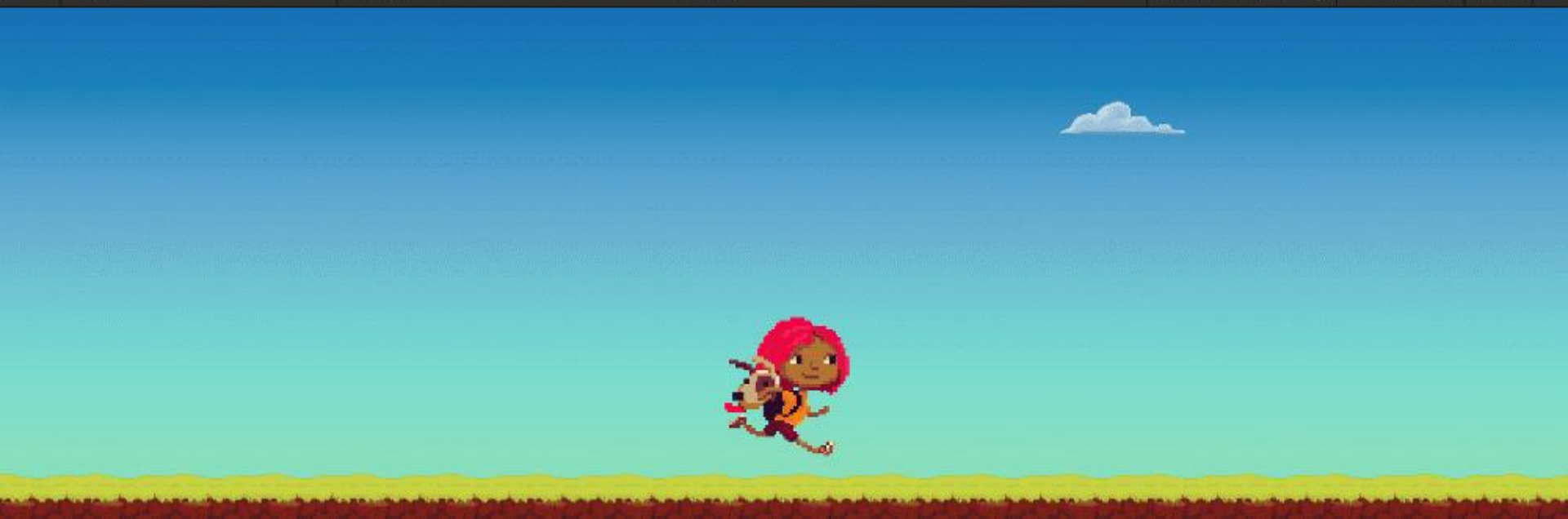
- Several modern game engines enable scripting through a **visual** language
- These are often called **blueprints**
- Takes simplicity a step further by allowing a **no-code** solution to dynamic features
- Trade-off: complete control of normal programming vs ease of visual drag/drop

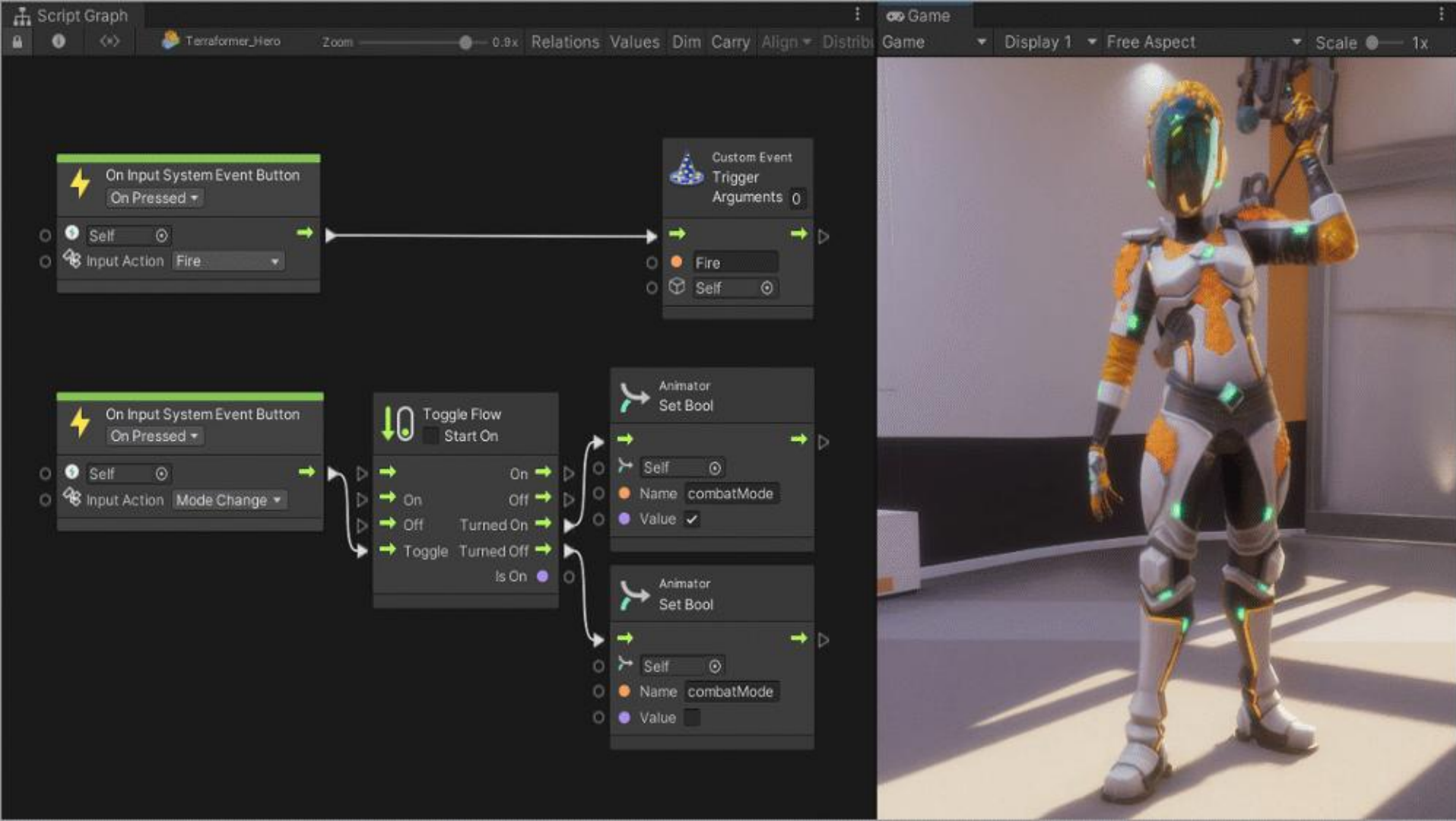


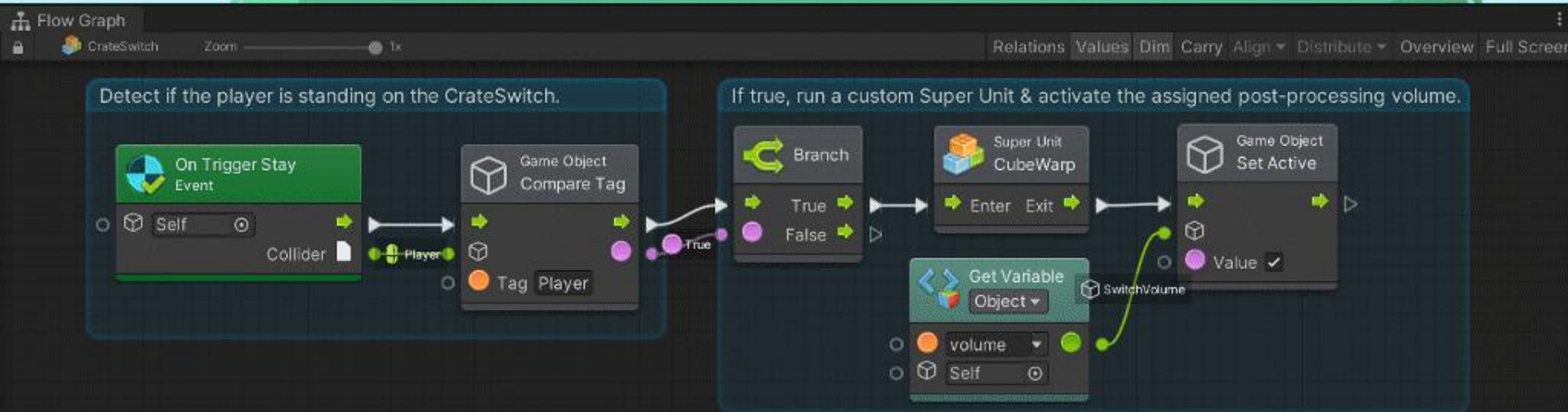


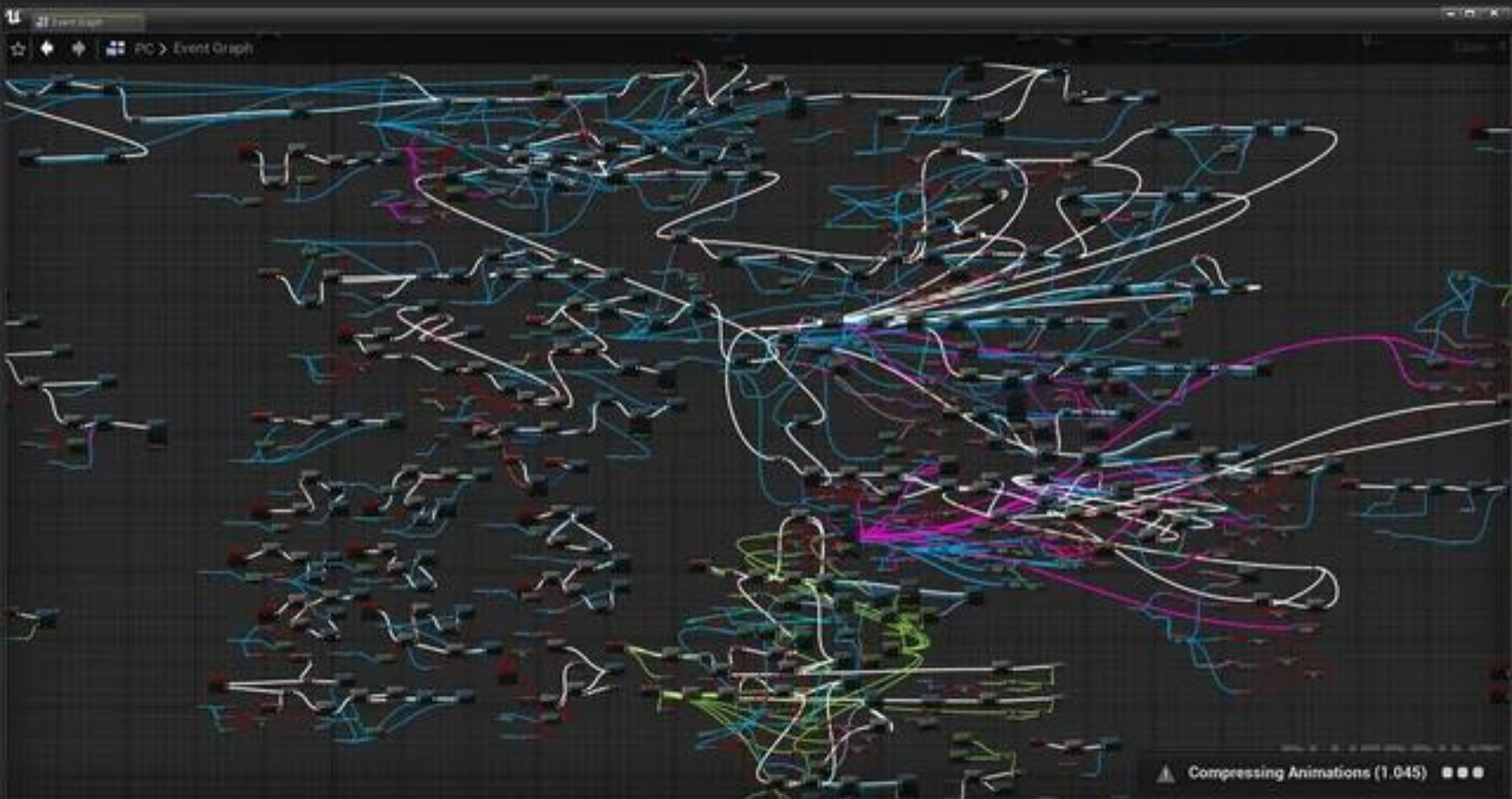


1 16:9 Scale 1.43x Maximize On Play Mute Audio Stats Gizmo









Lua Overview

- Created as a lightweight, embeddable language for use in many types of software, but especially for **games**
- Easy to learn for **beginners**, simple syntax
- Small **memory** footprint
- Lua **Tables** make **data** storage easy
- Garbage collected language

Hello, World

```
-- write code in hello.lua file
```

```
-- print to the console
```

```
print("Hello, World!")
```

```
-- run code with lua interpreter
```

```
lua hello.lua
```

```
-- Define variables
local name = "Lua"
local greeting = "Hello"

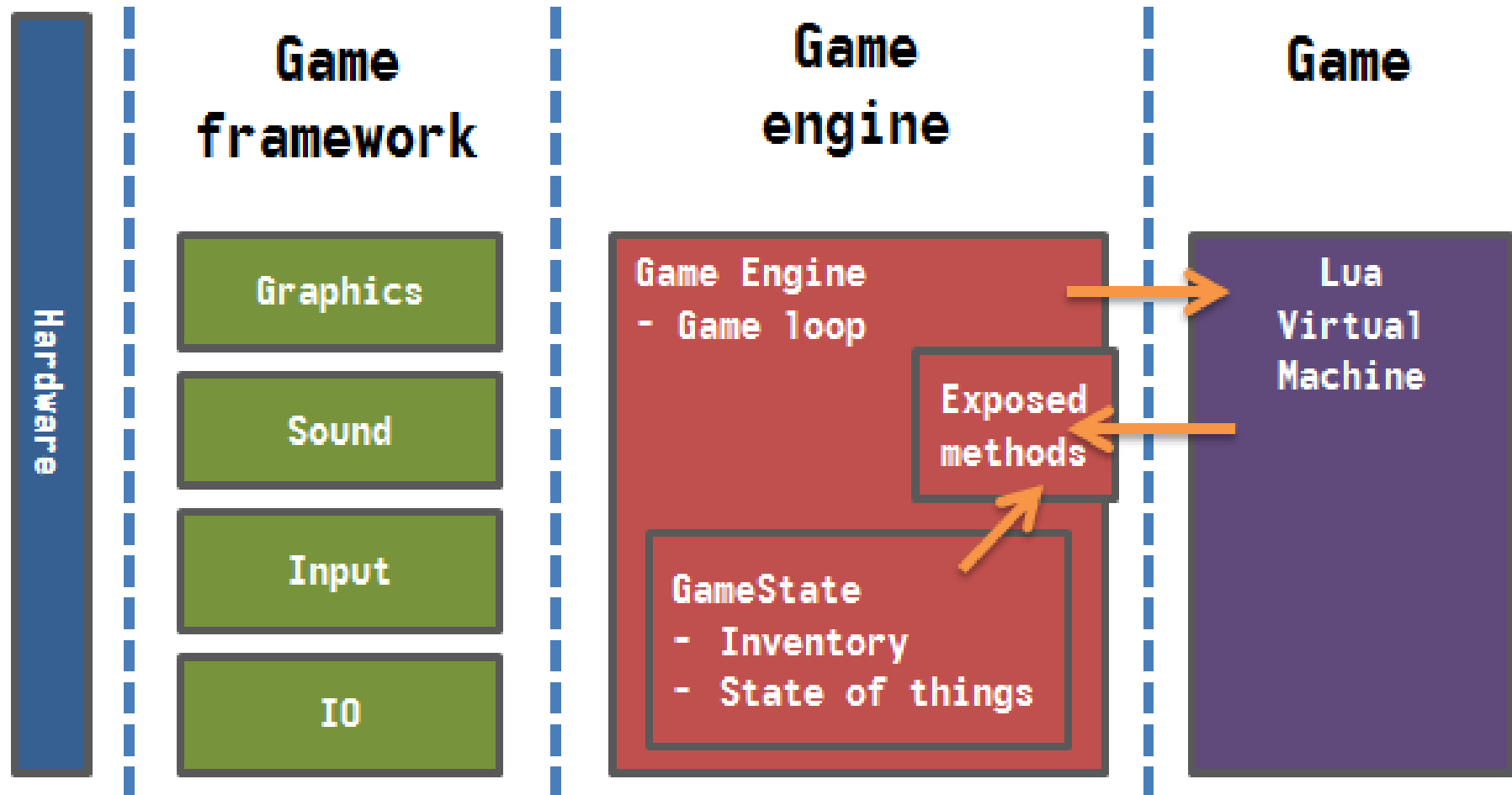
-- Define a table
local messages = {
    "Welcome to programming!",
    "Enjoy coding in " .. name .. "!",
}

-- Print the greeting
print(greeting .. ", " .. name .. "!")

-- Print messages from the table
for _, message in ipairs(messages) do
    print(message)
end
```

Integrating Lua with C++

- Lua can **integrate** with C++ by **including** the Lua interpreter in your C++ code
- You can execute Lua scripts by loading them from **files**, or from C++ **strings**
- C++ data and functions can be **exposed** to Lua, allowing scripts to call them
- C++ can also call Lua functions



Lua / C++ - Execute String

```
#include <lua.hpp>
```

```
int main()
```

```
{
```

```
    lua_State* L = luaL_newstate();
```

```
    luaL_dostring(L, "print('Hello, World!')");
```

```
    lua_close(L);
```

```
    return 0;
```

```
}
```


Lua / C++ - Execute File

```
#include <lua.hpp>
```

```
int main()  
{  
    lua_State* L = luaL_newstate();  
    luaL_dofile(L, "hello.lua");  
    lua_close(L);  
    return 0;  
}
```

Simple Error Handling

```
int main()
{
    lua_State* L = luaL_newstate();
    if (luaL_dofile(L, "hello.lua") != LUA_OK)
    {
        const char* error = lua_tostring(L, -1);
        printf("Error: %s\n", error);
        lua_pop(L, 1);
    }
    lua_close(L);
    return 0;
}
```

C++ / Lua Data Exchange

- Lua and C++ can **exchange** primitive data types for straightforward data **sharing**
- C++ can pass data to Lua
 - Example: Player Health, Score, etc
- Lua can execute code and return to C++
 - Example: Result of entity behavior / dialogue

```
int main()
{
    lua_State* L = luaL_newstate(); // Create a new Lua state
    luaL_openlibs(L);               // Open Lua standard libraries
    lua_pushnumber(L, 42);           // Push 42 onto the Lua stack
    lua_setglobal(L, "x");           // Make in Lua as var "x"
    luaL_dostring(L, "x = x * 2");   // Lua multiplies "x" by 2
    lua_getglobal(L, "x");           // Get the global "x" from Lua
    double r = lua_tonumber(L, -1);  // Convert to number
    lua_close(L);                    // Close the Lua state
    std::cout << "Result: " << r << "\n";
    return 0;
}
```

```
int main()
{
    lua_State* L = luaL_newstate(); // Create a new Lua state
    luaL_openlibs(L);               // Open Lua standard libraries

    std::map<std::string, int> cppMap = { {"one", 1}, {"two", 2}, {"three", 3} };

    lua_newtable(L); // Create a new table on the Lua stack

    for (const auto& pair : cppMap) {
        lua_pushstring(L, pair.first.c_str()); // Push the key (string)
        lua_pushinteger(L, pair.second);       // Push the value (integer)
        lua_settable(L, -3);                   // Set the key-value pair in the table
    }

    lua_setglobal(L, "cppTable"); // Set the table as a global variable in Lua
    luaL_dostring(L,
        "for k, v in pairs(cppTable) do "
        "    print(k, v) "
        "end"
    );

    lua_close(L); // Close the Lua state
    return 0;
}
```

Using Lua with ECS

- Our C++ game engine has been using the **ECS** architecture for all Entities
- It can be a little bit **cumbersome** at first to convert our Entities to a format that Lua can read and modify
- However, once this is done, it can be **used for all Lua scripts** involving entities

Entities to Lua

- There are two overall main ways to convert our Entities to Lua data
 1. Serialize Entity to a string that can be interpreted as a Lua table
 2. Go through Entity data and manually pass each variable to Lua

1. Serialization Example

```
{  
    id = 1,  
    tag = "enemy",  
    active = true,  
    components = {  
        transform = { x = 10, y = 20 },  
        lifespan = { remaining = 5, total = 10 },  
        health = { current = 50, max = 100 },  
    }  
}
```



```
std::string toLuaTable() const {
    std::ostringstream luaTable;
    luaTable << "{\n";
    luaTable << "    id = " << m_id << ",\n";
    luaTable << "    tag = \"" << m_tag << "\",\n";
    luaTable << "    active = " << (m_active ? "true" : "false") << ",\n";
    luaTable << "    components = {\n";

    if (has<CTransform>()) {
        const auto& transform = get<CTransform>();
        luaTable << "        transform = { x = " << transform.x
            << ", y = " << transform.y << " },\n";
    }

    if (has<CLifeSpan>()) {
        const auto& lifespan = get<CLifeSpan>();
        luaTable << "        lifespan = { remaining = " << lifespan.remaining
            << ", total = " << lifespan.total << " },\n";
    }

    if (has<CHealth>()) {
        const auto& health = get<CHealth>();
        luaTable << "        health = { current = " << health.current
            << ", max = " << health.max << " },\n";
    }

    luaTable << "    }\n";
    luaTable << "}";
    return luaTable.str();
}
```

```
void pushEntitiesToLua(lua_State* L, const EntityVec& entities) {  
    lua_newtable(L); // Create a new table on Lua stack  
  
    int index = 1; // Lua arrays are 1-indexed  
    for (const auto& entity : entities) {  
        lua_newtable(L); // Create a table for each entity  
  
        // Push transform data if the entity has CTransform  
        if (entity->has<CTransform>()) {  
            const auto& transform = entity->get<CTransform>();  
  
            lua_newtable(L); // Create a table for the transform  
            lua_pushnumber(L, transform.x);  
            lua_setfield(L, -2, "x");  
            lua_pushnumber(L, transform.y);  
            lua_setfield(L, -2, "y");  
            lua_setfield(L, -2, "transform"); // entity["transform"] = transform table  
        }  
  
        lua_rawseti(L, -2, index++); // entities[index] = entity table  
    }  
}
```

Modify Entities in Lua

```
-- iterate_entities.lua
function moveEntitiesToLeft(entities)
    for _, entity in ipairs(entities) do
        if entity.transform then
            -- Move the transform position to the left by 5
            entity.transform.x = entity.transform.x - 5
        end
    end
end
```

Getting Entity From Lua

- If we are using the Serialization approach to send a 'table string' to Lua, it makes sense to **reverse** this back to C++
- Lua can serialize the Entity and send it back to be read as a C++ Entity

```
void Entity::fromLuaTable(lua_State* L, const std::string& luaTableStr)
{
    // Load the Lua table string into Lua
    luaL_dostring(L, ("entity = " + luaTableStr).c_str());

    // Get the `entity` table from Lua
    lua_getglobal(L, "entity");

    // Update Entity ID, tag, and active state
    lua_getfield(L, -1, "id");
    m_id = static_cast<size_t>(lua_tointeger(L, -1));
    lua_pop(L, 1);

    lua_getfield(L, -1, "tag");
    m_tag = lua_tostring(L, -1);
    lua_pop(L, 1);

    lua_getfield(L, -1, "active");
    m_active = lua_toboolean(L, -1);
    lua_pop(L, 1);
}
```

```
// Update components
lua_getfield(L, -1, "components");

// Transform
lua_getfield(L, -1, "transform");
auto& transform = add<CTransform>();
lua_getfield(L, -1, "x");
transform.x = static_cast<float>(lua_tonumber(L, -1));
lua_pop(L, 1);

lua_getfield(L, -1, "y");
transform.y = static_cast<float>(lua_tonumber(L, -1));
lua_pop(L, 1);
lua_pop(L, 1);
```

```
// Health
lua_getfield(L, -1, "health");
auto& health = add<CHealth>();
lua_getfield(L, -1, "current");
health.current = static_cast<int>(lua_tointeger(L, -1));
lua_pop(L, 1);

lua_getfield(L, -1, "max");
health.max = static_cast<int>(lua_tointeger(L, -1));
lua_pop(L, 1);
lua_pop(L, 1);

// LifeSpan
lua_getfield(L, -1, "lifespan");
auto& lifespan = add<CLifeSpan>();
lua_getfield(L, -1, "remaining");
lifespan.remaining = static_cast<int>(lua_tointeger(L, -1));
lua_pop(L, 1);

lua_getfield(L, -1, "total");
lifespan.total = static_cast<int>(lua_tointeger
```

ECS and Lua

- While it seems like a lot of work to have to convert ECS Entities to Lua, you **only need to do it once** in the C++ engine
- Since all game logic revolves around the same **Entity** objects, all the Lua code will re-use this same Entity table structure
- Same code can be **re-used** in diff systems

Dynamic Game Events with Lua

- We can now create a library of Lua files that can be called when various events occur in our game engine
 - Ex: Cutsscenes, timed effects, npc behaviours
- **Triggered events** are a common way to implement many behaviors in games
 - Ex: When Entity enters a predefined area of the game, call a script on that Entity

Events / Triggers

- Entities can be given various triggered events that evoke **specific functions**
- Entity Event Examples:
 - onEnter
 - onExit
 - onHit
 - onDeath
 - onSpawn

onEnter example

```
function onEnter(entity, otherEntity)
  otherEntity.poison = {
    duration = 5,          -- Poison lasts for 5 seconds
    damagePerSecond = 10  -- Deals 10 damage per second
  }
end
```

```
function onEnter(entity, otherEntity, quest)
  quest.enteredLocation = true;
end
```

onDeath() example

```
function onDeath(entity, entities)
  -- Check if the entity has a loot component
  if entity.loot then
    for _, lootItem in ipairs(entity.loot.items) do
      -- Roll for the loot probability
      if math.random() < lootItem.probability then
        -- Create a new entity for the loot item
        local lootEntity = {
          type = "loot",
          transform = {
            x = entity.transform.x + math.random(-10, 10),
            y = entity.transform.y + math.random(-10, 10)
          },
          loot = {
            itemId = lootItem.id
          }
        }
        -- Add the loot entity to the entities list
        table.insert(entities, lootEntity)
      end
    end
  end
end
```

Performance Considerations

- Moving data between C++ and Lua is obviously **slower** than staying in C++
- This is why we keep computation heavy code in the engine and use outside scripting only for **very simple** tasks
- We are trading the performance loss for the **convenience** of Lua scripting

Performance Considerations

- There is an **overhead** associated with each separate call to Lua from C++
- We want to make **as few calls** to Lua as possible from the C++ engine
- For example, make one call on **all Entities** rather than one call for each Entity
- **Preload** Lua scripts in advance

C++ / Lua Videos

- “Embedding Lua in C++” Playlist
 - <https://www.youtube.com/playlist?list=PLLwK93hM93Z3nhfJyRRWGRXHaXgNX0Itk>
- “Embedding Lua in C++ #1”
 - <https://www.youtube.com/watch?v=4l5HdmPoynw>
- “Automate sequences via Lua...”
 - <https://www.youtube.com/watch?v=E42Lyv2Ra1c>
- “Lua for C++ Programmers”
 - <https://www.youtube.com/watch?v=pfwHCiP1HFM>